HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

OFFICE FOR INTERNATIONAL STUDY PROGRAM

SEMESTER: 261

◇ ··· ◇



MICROPROCESSORS-MICROCONTROLLERS

LAB 3 REPORT

INSTRUCTOR: MR. PHAN VAN SY

Student Name: Du Duc Minh

Student ID: 2452734

Class ID: CC03

Ho Chi Minh City, November 02, 2026

Contents

1 Exercise 1 **3**

2 Exercise 2 **3**

3 Exercise 3 **4**

4 Exercise 4 **4**

5 Exercise 5 **6**

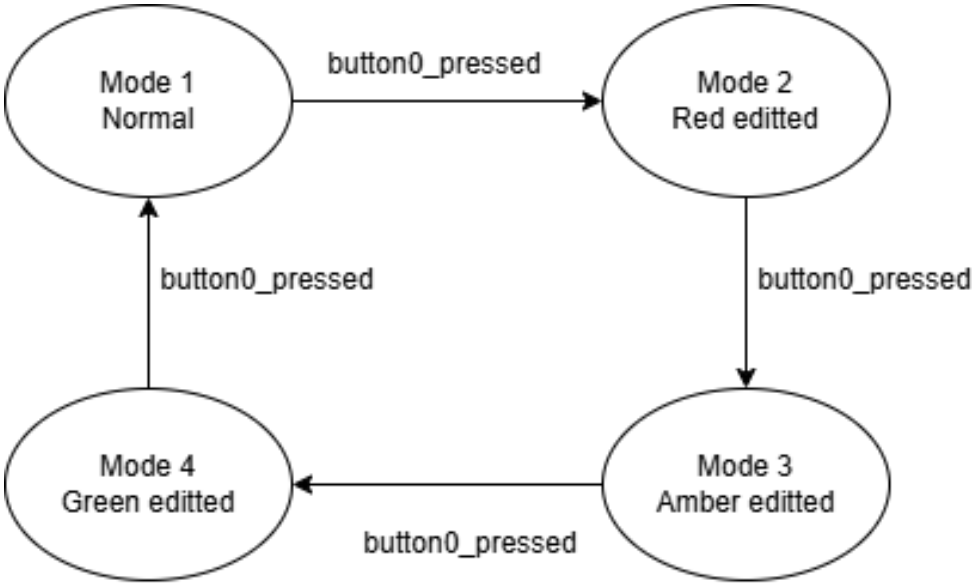
6 Exercise 6 **8**

7 Exercise 7-8-9 **11**

8 Exercise 10 **13**

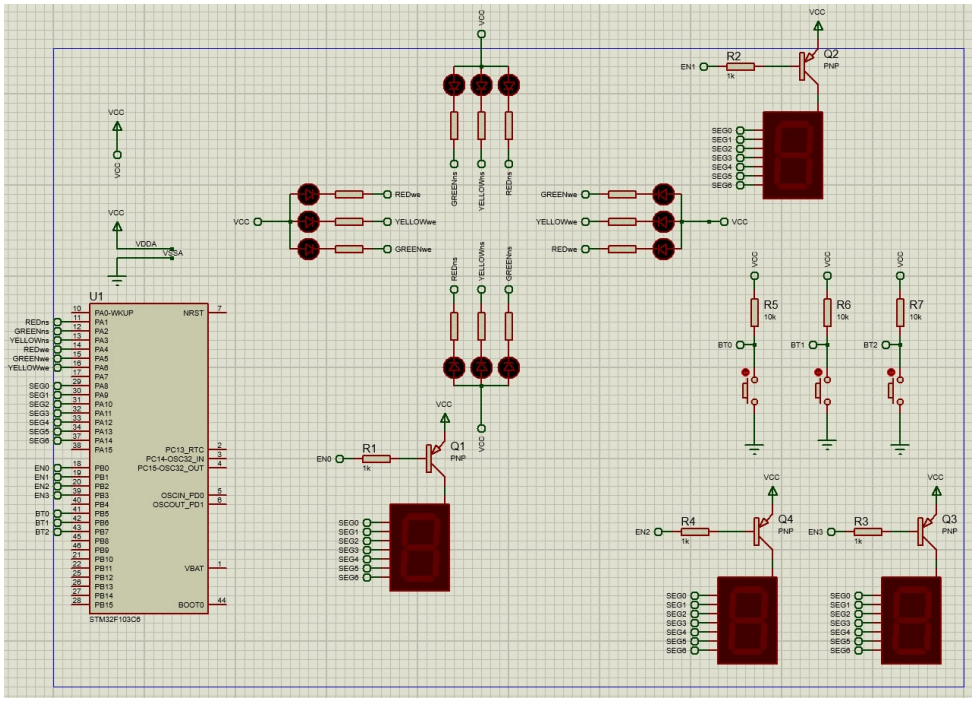
1 Exercise 1

Schematic from Proteus for Exercise 1:



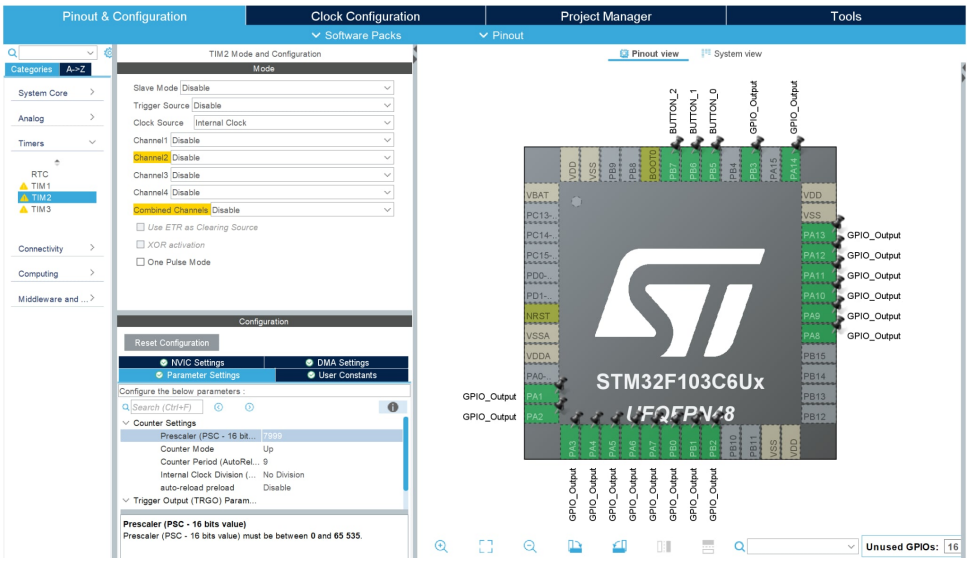
2 Exercise 2

Schematic from Proteus for Exercise 2:



3 Exercise 3

Create project that has pin corresponding to proteus schematic:



4 Exercise 4

In this exercise I implement software timer functions to control and generate clock for system.

Source code for software timer

```
static volatile uint32_t timer0_counter, timer1_counter,
    timer2_counter, freq_counter;
volatile int timer0_flag, timer1_flag, timer2_flag, freq_flag;

static void set_timer(volatile uint32_t *counter, volatile int *
    flag, int duration)
{
    uint32_t irq_state = __get_PRIMASK();
    __disable_irq();
    /* Positive durations round up; unarmed timers never raise a
       flag. */
```

```
9       *counter = duration > 0 ? ((uint32_t)duration + TIMER_CYCLE -
            1U) / TIMER_CYCLE : 0U;
10      *flag = duration <= 0;
11      __set_PRIMASK(irq_state);
12  }
13
14  void setTimer0(int duration) { set_timer(&timer0_counter, &
        timer0_flag, duration); }
15  void setTimer1(int duration) { set_timer(&timer1_counter, &
        timer1_flag, duration); }
16  void setTimer2(int duration) { set_timer(&timer2_counter, &
        timer2_flag, duration); }
17  void setTimerFreq(int duration) { set_timer(&freq_counter, &
        freq_flag, duration); }
18
19  static void tick_timer(volatile uint32_t *counter, volatile int *
        flag)
20  {
21      if (*counter > 0U && --(*counter) == 0U) *flag = 1;
22  }
23
24  void timer_run(void)
25  {
26      tick_timer(&timer0_counter, &timer0_flag);
27      tick_timer(&timer1_counter, &timer1_flag);
28      tick_timer(&timer2_counter, &timer2_flag);
29      tick_timer(&freq_counter, &freq_flag);
30  }
```

5 Exercise 5

This exercise focuses on correctly catching the button input signal. The problem of reading the button is the bounce of the button when people press it. I implement the input_reading.h/c file to deal with this problem. The core idea of this file it only catch the input signal that stay and still stay more than 10ms (legal signal).

Source code for input reading

```
static const ButtonConfig button[NO_OF_BUTTONS] = {
    {BUTTON_0_GPIO_Port, BUTTON_0_Pin},
    {BUTTON_1_GPIO_Port, BUTTON_1_Pin},
    {BUTTON_2_GPIO_Port, BUTTON_2_Pin}
};
/* Active-low inputs must start RELEASED, not at C's default zero
   . */
static volatile GPIO_PinState buttonBuffer[NO_OF_BUTTONS];
static GPIO_PinState debounceButtonBuffer1[NO_OF_BUTTONS];
static GPIO_PinState debounceButtonBuffer2[NO_OF_BUTTONS];
static volatile uint8_t flagForButtonPress1s[NO_OF_BUTTONS];
static uint16_t counterForButtonPress1s[NO_OF_BUTTONS];

/* 100 samples at the default 10 ms period, as in your original
   code. */
#define DURATION_FOR_AUTO_INCREASING \
    ((1000U + BUTTON_SAMPLE_PERIOD_MS - 1U) /
       BUTTON_SAMPLE_PERIOD_MS)

void button_init(void)
{
    for (uint8_t i = 0; i < NO_OF_BUTTONS; ++i) {
        buttonBuffer[i] = GPIO_PIN_SET;
        debounceButtonBuffer1[i] = GPIO_PIN_SET;
```

```
22          debounceButtonBuffer2[i] = GPIO_PIN_SET;
23          flagForButtonPress1s[i] = 0;
24          counterForButtonPress1s[i] = 0;
25      }
26  }
27  /* Called once per sample from TIM2, never again from the main
       loop. */
28  void button_reading(void)
29  {
30      for (uint8_t i = 0; i < NO_OF_BUTTONS; ++i) {
31          GPIO_PinState old_state = buttonBuffer[i];
32          debounceButtonBuffer2[i] = debounceButtonBuffer1[i];
33          debounceButtonBuffer1[i] = HAL_GPIO_ReadPin(button[i].
               port, button[i].pin);
34
35          if (debounceButtonBuffer1[i] == debounceButtonBuffer2[i])
                {
36              buttonBuffer[i] = debounceButtonBuffer1[i];
37          }
38
39          if (buttonBuffer[i] == GPIO_PIN_RESET) {
40              /* Start at zero on the accepted press; then count
                   full intervals. */
41              if (old_state == GPIO_PIN_SET) {
42                  counterForButtonPress1s[i] = 0;
43              } else if (counterForButtonPress1s[i] <
                   DURATION_FOR_AUTO_INCREASING) {
44                  counterForButtonPress1s[i]++;
45              }
46              flagForButtonPress1s[i] =
47                  counterForButtonPress1s[i] >=
                       DURATION_FOR_AUTO_INCREASING;
```

```
48          } else {
49              counterForButtonPress1s[i] = 0;
50              flagForButtonPress1s[i] = 0;
51          }
52      }
53  }
```

Button reading is call inside timer interrupt

```
1   void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
2   {
3       static uint32_t button_sample_counter;
4       if (htim->Instance == TIM2) {
5           /* At the default 10 ms period, read buttons on every
               interrupt.
6            * If TIM2 is faster, count ticks to keep samples 10 ms
                apart. */
7           if (++button_sample_counter >= BUTTON_SAMPLE_TICKS) {
8               button_sample_counter = 0;
9               button_reading();
10          }
11          timer_run();
12      }
13  }
```

6 Exercise 6

For 4 7-segment Leds, I create a led buffer contains 4 spaces to store the corresponded value displayed on led. This buffer is modified by mode and FSM controller. In normal mode, it just enable 2 7-segment leds that use for display counter of each road. Conversely, in non-normal mode, the others 2 leds is enable, one display the current mode and one display the temporal value that I want to set.

Source code for display 7-segment leds buffer

```
#define SEGMENT_PINS (GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10 | GPIO_PIN_11 | \
                      GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14)
#define ENABLE_PINS (GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3)
#define TRAFFIC_PINS (GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3 | \
                      GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_6)
int led_buffer[4];
static uint8_t scan_index;
void led_display_init(void)
{
    HAL_GPIO_WritePin(GPIOB, ENABLE_PINS, GPIO_PIN_SET);
    clearAllLeds();
    traffic_display(LED_OFF, LED_OFF);
    for (uint8_t i = 0; i < 4U; ++i) {
        led_buffer[i] = -1;
    }
    scan_index = 0;
    setTimerFreq(10);
}

void led_display_normal(uint8_t west, uint8_t north)
{
    /* Each enable controls an independent display in the schematic. */
    led_buffer[0] = west;   /* EN0 / PB0: west's currently lit color. */
    led_buffer[1] = north;  /* EN1 / PB1: north's currently lit color. */
    led_buffer[2] = -1;
    led_buffer[3] = -1;
```

```
}

void led_display_edit(uint8_t mode, uint8_t value)
{
    led_buffer[0] = -1;
    led_buffer[1] = -1;
    led_buffer[2] = mode;   /* EN2 / PB2: mode 2, 3, or 4. */
    led_buffer[3] = value;  /* EN3 / PB3: temporary, unsaved
       duration. */
}
```

There is a function used to control the enable of 4 7-segment leds. These leds are designed to turn on and turn off sequentially. Therefore, I have a freq_timmer to enable the switch activity of 7-segment leds.

Source code for display traffic lights

```
void led_display_refresh(void)
{
    if (!freq_flag) return;
    setTimerFreq(10);
    HAL_GPIO_WritePin(GPIOB, ENABLE_PINS, GPIO_PIN_SET);
    /* Only two displays are used in each mode. Skip blank slots
       so each
     * active display refreshes every 20 ms with the default 10
       ms tick. */
    for (uint8_t checked = 0; checked < 4U; ++checked) {
        if (led_buffer[scan_index] >= 0 && led_buffer[scan_index]
            <= 9) break;
        scan_index = (scan_index + 1U) % 4U;
    }
    int digit = led_buffer[scan_index];
    display7SEG(digit);
```

```
14      if (digit >= 0 && digit <= 9) {
15          HAL_GPIO_WritePin(GPIOB, (uint16_t)(GPIO_PIN_0 <<
                scan_index), GPIO_PIN_RESET);
16      }
17      scan_index = (scan_index + 1U) % 4U;
18  }
```

For traffic lights displayed, I create a function had two input argument that show exactly which led will be displayed. The two input arguments here is controlled inside the fsm controller.

Source code for display traffic lights

```
1   void traffic_display(TrafficColor north_south, TrafficColor
        west_east)
2   {
3       static const uint16_t ns_pins[] = {0, GPIO_PIN_1, GPIO_PIN_3,
            GPIO_PIN_2};
4       static const uint16_t we_pins[] = {0, GPIO_PIN_4, GPIO_PIN_6,
            GPIO_PIN_5};
5       uint16_t on_pins = ns_pins[north_south] | we_pins[west_east];
6       /* One atomic port write avoids a transient wrong color.
           Active low. */
7       GPIOA->BSRR = (uint32_t)(TRAFFIC_PINS & ~on_pins) | ((
           uint32_t)on_pins << 16U);
8   }
```

7 Exercise 7-8-9

These exercise required making the activities of second and third button. I create two functions that will be called when each button is pressed. The logic of increment value function is just increment value of temporal variable by 1, if it reach the limit, it return to 1 when a new signal pass to it. Then for save value function, the logic is save the temporal

value into value buffer when the function is called. Beside, there is another feature of this function is the timer of red led always equal to the sum of timer of others two led, this feature make the traffic light system more realistic in real life.

Source code for increment temporal value

```
static void increment_value(void)
{
    if (mode == NORMAL) return;
    edit_value = edit_value >= MAX_DURATION ? 1U : edit_value + 1
       U;
    invalid_value = 0;
}
```

Source code for save the temporal value into buffer

```
static void save_value(void)
{
    if (mode == NORMAL) return;
    if (edit_value < 1U || edit_value > MAX_DURATION) {
        invalid_value = 1;
        return;
    }

    /* Keep both roads consistent: red = green + amber.
     * A red edit adjusts green; a green/amber edit adjusts red.
     * Reject impossible combinations without changing any saved
       values. */
    if (mode == MODIFY_RED) {
        if (edit_value <= value_buffer[AMBER_TIME]) {
            invalid_value = 1;
            return;
        }
```

```
17          value_buffer[RED_TIME] = edit_value;
18          value_buffer[GREEN_TIME] = edit_value - value_buffer[
                AMBER_TIME];
19      } else {
20          uint8_t index = mode - MODIFY_RED;
21          uint8_t other = index == AMBER_TIME ? GREEN_TIME :
                AMBER_TIME;
22          uint16_t red_time = (uint16_t)edit_value + value_buffer[
                other];
23          if (red_time > MAX_DURATION) {
24              invalid_value = 1;
25              return;
26          }
27          value_buffer[index] = edit_value;
28          value_buffer[RED_TIME] = (uint8_t)red_time;
29      }
30      invalid_value = 0;
31  }
```

8 Exercise 10

Firstly, I create run normal function, this function is active when mode is normal, it generate value passed to display traffic light.

Source code for control the activities of leds in normal mode

```
1   static void run_normal(void)
2   {
3       /* Timer 0 expires once per second; its setter also clears
           the flag. */
4       if (timer0_flag) {
5           setTimer0(1000);
```

```
        if (--phase_remaining == 0U) {
            phase = (enum TrafficPhase)((phase + 1U) % 4U);
            phase_remaining = (phase == NS_RED_WE_AMBER || phase
               == NS_AMBER_WE_RED)
                                ? value_buffer[AMBER_TIME] :
                                   value_buffer[GREEN_TIME];
        }
    }
    switch (phase) {
    case NS_RED_WE_GREEN:
        traffic_display(LED_RED, LED_GREEN);
        led_display_normal(phase_remaining, phase_remaining +
           value_buffer[AMBER_TIME]);
        break;
    case NS_RED_WE_AMBER:
        traffic_display(LED_RED, LED_AMBER);
        led_display_normal(phase_remaining, phase_remaining);
        break;
    case NS_GREEN_WE_RED:
        traffic_display(LED_GREEN, LED_RED);
        led_display_normal(phase_remaining + value_buffer[
           AMBER_TIME], phase_remaining);
        break;
    case NS_AMBER_WE_RED:
        traffic_display(LED_AMBER, LED_RED);
        led_display_normal(phase_remaining, phase_remaining);
        break;
    }
}
```

This function is the fsm correspond to mode of this system. When the legal input button trigger, the mode is change to the next mode in sequentially. Then, for normal mode, the system just call run normal to display 2 way road traffic light normally, the

input of button1 and button2 now is ignored. In non-normal mode, the system can receive the legal input of button1 to increment the temporal value buffer and receive the legal input of button2 to save the temporal value to the real value buffer. The traffic light led is blink by on software timer at 2Hz frequency, the color of led blinked is suitable to current mode.

Source code for fsm

```
static void switch_mode(uint8_t increment_held)
{
    mode = mode == MODIFY_GREEN ? NORMAL : (enum Mode)(mode + 1);
    blink_on = 1;
    invalid_value = 0;
    /* A held increment button must be released before editing a
       new mode. */
    suppress_increment = increment_held;
    if (mode == NORMAL) start_normal();
    else {
        edit_value = value_buffer[mode - MODIFY_RED];
        setTimer1(250);
    }
}
void fsm_for_input_processing(void)
{
    uint8_t pressed[NO_OF_BUTTONS], held[NO_OF_BUTTONS];
    uint8_t press_event[NO_OF_BUTTONS] = {0};
    uint8_t repeat_event = 0;
    uint32_t irq_state = __get_PRIMASK();
    /* Read one coherent snapshot of the interrupt-owned input
       buffers. */
    __disable_irq();
    for (uint8_t i = 0; i < NO_OF_BUTTONS; ++i) {
        pressed[i] = is_button_pressed(i);
```

```
24          held[i] = is_button_pressed_1s(i);
25      }
26      __set_PRIMASK(irq_state);
27
28      if (!pressed[1]) suppress_increment = 0;
29      for (int i = 0; i < (int)NO_OF_BUTTONS; i++) {
30          switch (buttonState[i]) {
31          case BUTTON_RELEASED:
32              if (pressed[i]) {
33                  buttonState[i] = BUTTON_PRESSED;
34                  press_event[i] = 1;
35              }
36              break;
37          case BUTTON_PRESSED:
38              if (!pressed[i]) buttonState[i] = BUTTON_RELEASED;
39              else if (held[i]) {
40                  buttonState[i] =
                        BUTTON_PRESSED_MORE_THAN_1_SECOND;
41                  if (i == 1U) {
42                      repeat_event = 1;
43                      setTimer2(500);
44                  }
45              }
46              break;
47          case BUTTON_PRESSED_MORE_THAN_1_SECOND:
48              if (!pressed[i]) buttonState[i] = BUTTON_RELEASED;
49              else if (i == 1U && timer2_flag) {
50                  setTimer2(500);
51                  repeat_event = 1;
52              }
53              break;
54          }
```

```
    }

    /* Mode has priority if several buttons are pressed together.
        */
    if (press_event[0]) switch_mode(pressed[1]);
    else {
        if (!suppress_increment && (press_event[1] ||
           repeat_event)) increment_value();
        if (press_event[2]) save_value();
    }

    if (mode == NORMAL) run_normal();
    else {
        /* 2 Hz = 500 ms per complete cycle = toggle every 250 ms
           . */
        if (timer1_flag) {
            setTimer1(250);
            blink_on = !blink_on;
        }
        TrafficColor color = (TrafficColor)(mode - MODIFY_RED +
           LED_RED);
        traffic_display(blink_on ? color : LED_OFF, blink_on ?
           color : LED_OFF);
        led_display_edit(mode, edit_value);
        /* Rejected save: blink EN3; the mode on EN2 stays
           visible. */
        if (invalid_value && !blink_on) led_buffer[3] = -1;
    }
}
```

Source code for Initial state of system

```
static void start_normal(void)
```

```
{
    phase = NS_RED_WE_GREEN;
    phase_remaining = value_buffer[GREEN_TIME];
    setTimer0(1000);
}
void input_processing_init(void)
{
    for (uint8_t i = 0; i < NO_OF_BUTTONS; ++i) buttonState[i] =
       BUTTON_RELEASED;
    value_buffer[RED_TIME] = 5;
    value_buffer[AMBER_TIME] = 2;
    value_buffer[GREEN_TIME] = 3;
    mode = NORMAL;
    edit_value = invalid_value = suppress_increment = 0;
    blink_on = 1;
    start_normal();
}
```

GitHub Repository

My Github repository url: https://github.com/Minh-0405/BKU_Lab_MPU_MCU